Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №19

Дисциплина: Основы программирования на Python

Тема: Микросервисная архитектура Python-приложений: взаимодействие сервисов и паттерны проектирования

Вариант 2: Библиотека

Цель работы

Получить практические навыки проектирования и разработки приложений с микросервисной архитектурой. В процессе выполнения работы необходимо понять отличие монолитной и микросервисной архитектуры, научиться выделять отдельные сервисы из предметной области, организовывать взаимодействие микросервисов, использовать REST API для синхронного взаимодействия, познакомиться с асинхронным взаимодействием через брокер сообщений, реализовать API Gateway, использовать отдельную базу данных для каждого сервиса, применить паттерны проектирования микросервисных систем, реализовать обработку временной недоступности сервисов и использовать Docker Compose для запуска распределённого приложения.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать распределённое приложение библиотеки, состоящее не менее чем из трёх микросервисов: сервис читателей, сервис книг и сервис выдачи книг. Каждый сервис имеет собственную зону ответственности, предоставляет REST API, использует собственное хранилище данных, запускается независимо и упаковывается в Docker-контейнер. Дополнительно требуется реализовать API Gateway как единую точку входа и обеспечить проверку доступности книги при выдаче с регистрацией события выдачи.

Краткие теоретические сведения

В монолитном приложении все компоненты собраны в одном процессе: это упрощает разработку и развёртывание, но затрудняет независимое масштабирование и приводит к тому, что сбой одной части останавливает всю систему. Микросервисная архитектура делит приложение на небольшие сервисы, каждый из которых отвечает за одну предметную область, развёртывается отдельно и взаимодействует с остальными по сети.

Синхронное взаимодействие выполняется по REST API: сервис отправляет HTTP-запрос и ожидает ответ, что удобно для проверок и запросов данных. Асинхронное взаимодействие строится через брокер сообщений: сервис публикует событие и не ждёт его обработки, а подписчик обрабатывает событие в удобное время. Такой подход снижает связность и повышает отказоустойчивость.

Паттерн API Gateway предполагает единую точку входа, которая маршрутизирует запросы клиента к нужным сервисам, выполняет агрегацию ответов и скрывает внутреннюю топологию. Паттерн Database per Service требует, чтобы каждый сервис владел собственными данными и не обращался к базе другого сервиса напрямую. Паттерны Retry и Circuit Breaker повышают устойчивость к временным сбоям: первый повторяет запрос с задержкой, второй прекращает обращения к неисправному сервису на время восстановления.

Docker Compose описывает многоконтейнерное приложение в одном файле: сервисы, образы, переменные окружения, тома, сети и проверки работоспособности. Запуск выполняется командой docker compose up, при этом контейнеры видят друг друга по именам сервисов внутри общей сети.

Обоснование выделения микросервисов

Предметная область библиотеки естественно делится на три части, различающиеся частотой изменений и характером нагрузки.

Сервис	Зона ответственности	Обоснование выделения
readers-service	Читатели: регистрация, проверка статуса, изменение данных	Справочные данные, редко изменяются, используются при каждой выдаче
books-service	Книги: каталог, наличие экземпляров, уменьшение и увеличение фонда	Наиболее нагруженная часть, требует отдельного масштабирования при поиске
loans-service	Выдачи: оформление, возврат, оркестрация проверок	Содержит бизнес-процесс, зависит от двух других сервисов

Такое разделение позволяет независимо изменять каталог книг и справочник читателей, а также масштабировать сервис книг при росте числа поисковых запросов. Сервис выдач выступает оркестратором: он не хранит сведения о книгах и читателях, а запрашивает их по сети.

Архитектура системы

Клиент обращается только к API Gateway на порту 8000. Gateway маршрутизирует запросы к сервисам читателей (8001), книг (8002) и выдач (8003). Каждый сервис хранит данные в собственном JSON-файле, а сервис выдач дополнительно публикует события в файловую шину, откуда их забирает отдельный потребитель.

Компонент	Порт	API	Хранилище
gateway	8000	Маршрутизация, агрегация, проверка системы	Не имеет собственного хранилища
readers-service	8001	GET, POST, PUT, DELETE /readers, GET /readers/check/{id}	data/readers.json
books-service	8002	CRUD /books, POST /books/{id}/decrement и increment, GET /books/{id}/availability	data/books.json
loans-service	8003	GET, POST /loans, POST /loans/{id}/return, GET /loans/events	data/loans.json и каталог data/events

Каждый сервис реализован на FastAPI и имеет собственные слои: маршруты, схемы Pydantic, сервис и репозиторий поверх общего класса JsonStorage. Общий код вынесен в пакет shared — хранилище, устойчивый HTTP-клиент и файловая шина событий, что позволяет не дублировать инфраструктурные решения.

Взаимодействие сервисов

Сценарий выдачи книги демонстрирует синхронное взаимодействие трёх сервисов. Сервис выдач сначала проверяет читателя запросом к сервису читателей, затем запрашивает сведения о книге и её доступности у сервиса книг, уменьшает число доступных экземпляров, создаёт запись о выдаче и публикует событие loan_issued. При возврате порядок обратный: закрывается выдача, увеличивается число доступных экземпляров и публикуется событие loan_returned. Если на любом шаге сервис недоступен, операция не считается выполненной и клиент получает код 503 с пояснением.

API Gateway реализует четыре паттерна: маршрутизацию запросов, агрегацию (сценарий /demo/borrow-book проверяет читателя и создаёт выдачу одним запросом клиента), сокрытие топологии и проверку состояния системы через /health/system, которая опрашивает маршрут /health каждого сервиса и возвращает сводку.

Листинг 19.1 — Проверка состояния системы в Gateway (gateway/app.py)

```
@app.get("/health/system")
def health():
    services = {"readers": READERS_URL, "books": BOOKS_URL,
                "loans": LOANS_URL}
    result = {}
    with httpx.Client(timeout=1.5) as client:
        for name, url in services.items():
            try:
                r = client.get(f"{url}/health")
                result[name] = ("[ok] healthy" if r.status_code == 200
                                else f"[fail] status={r.status_code}")
            except Exception:
                result[name] = "[fail] unavailable"
    return {"system": "healthy", "services": result}
```

Отказоустойчивость: Retry и Circuit Breaker

Для обращения к другим сервисам реализован клиент ResilientClient, объединяющий два паттерна. Retry выполняет до трёх попыток с экспоненциально растущей задержкой (0.3, 0.6 и 1.2 секунды) и повторяет запрос только при сетевых ошибках и кодах 502, 503, 504, поскольку остальные ошибки повтором не исправляются.

Circuit Breaker имеет три состояния. В состоянии CLOSED запросы проходят, а ошибки накапливаются; при достижении порога предохранитель переходит в OPEN и отклоняет запросы без обращения к сервису, что экономит время ожидания. По истечении периода восстановления он переходит в HALF-OPEN и пропускает один пробный запрос: успех возвращает систему в CLOSED, ошибка снова открывает предохранитель. Параметры порога и периода задаются переменными окружения CIRCUIT_FAILURES и CIRCUIT_TIMEOUT.

Листинг 19.2 — Circuit Breaker (shared/resilient_client.py)

```
class CircuitBreaker:
    def __init__(self, failure_threshold: int = 3,
                 recovery_timeout: float = 5.0, name: str = "service"):
        self._failure_threshold = failure_threshold
        self._recovery_timeout = recovery_timeout
        self._failures = 0
        self._opened_at = None
        self._state = CircuitBreakerState.CLOSED

    def _maybe_half_open(self):
        if self._state is CircuitBreakerState.OPEN and self._opened_at:
            if time.monotonic() - self._opened_at >= self._recovery_timeout:
                self._state = CircuitBreakerState.HALF_OPEN

    def before_request(self):
        self._maybe_half_open()
        if self._state is CircuitBreakerState.OPEN:
            raise CircuitBreakerError(
                f"[cb {self._name}] предохранитель открыт, "
                f"запрос отклонён без обращения к сервису")

    def on_success(self):
        self._state = CircuitBreakerState.CLOSED
        self._failures = 0
        self._opened_at = None

    def on_error(self):
        self._failures += 1
        if (self._state is CircuitBreakerState.HALF_OPEN
                or self._failures >= self._failure_threshold):
            self._state = CircuitBreakerState.OPEN
            self._opened_at = time.monotonic()
            self._failures = 0
```

Асинхронное взаимодействие

События публикуются в файловую шину FileEventBus, которая имитирует брокер сообщений: событие с уникальным идентификатором, типом, отметкой времени и полезной нагрузкой сохраняется в отдельный JSON-файл каталога events. Такой подход соответствует паттерну Transactional Outbox: публикация

отделена от бизнес-операции и не нарушает её атомарность, а при недоступности потребителя события не теряются.

Потребитель event_delivery/consumer.py периодически опрашивает каталог, обрабатывает новые события и продвигает указатель последнего обработанного идентификатора. Ошибка обработки одного события не останавливает цикл: она журналируется, а обработка продолжается со следующего события.

Контейнеризация и запуск

Все сервисы собираются из общего Dockerfile и запускаются через Docker Compose. Описаны четыре контейнера, общая сеть library_net, именованный том library-data для файловых хранилищ и проверки работоспособности healthcheck, опрашивающие маршрут /health каждого сервиса с интервалом десять секунд. Зависимость сервиса выдач от сервисов читателей и книг задана через условия запуска, поэтому порядок старта соблюдается автоматически. Запуск выполняется командой docker compose up --build, после чего вся система доступна на порту 8000.

Раздел тестирования

Тесты разделены по сервисам и включают модульные проверки логики, интеграционные проверки HTTP-маршрутов и проверки взаимодействия с подменёнными клиентами других сервисов.

Файл тестов	Кол-во	Что проверяется
books-service/tests/test_books.py	11	Создание и чтение книги, проверка доступности, уменьшение и увеличение фонда, отказ при отсутствии экземпляров, интеграционные вызовы через HTTP, health-check
readers-service/tests/test_readers.py	11	Создание и чтение читателя, дубликат электронной почты, изменение и удаление, проверка статуса, health-check и ответ 404
loans-service/tests/test_loans.py	12	Создание и список выдач, отсутствующая книга, недоступная книга, возврат, публикация события, интеграционный вызов через HTTP, порядок оркестрации, недоступность сервиса, открытие предохранителя

Все тридцать четыре теста проходят успешно. Отдельно проведена экспериментальная проверка отказоустойчивости: сервис книг останавливался, после чего запрос на выдачу возвращал код 503 с сообщением о недоступности; после нескольких неудачных попыток предохранитель переходил в состояние OPEN и последующие запросы отклонялись мгновенно, без ожидания тайм-аута. После запуска сервиса и истечения периода восстановления предохранитель переходил в HALF-OPEN, пробный запрос завершался успешно, и система возвращалась в рабочее состояние.

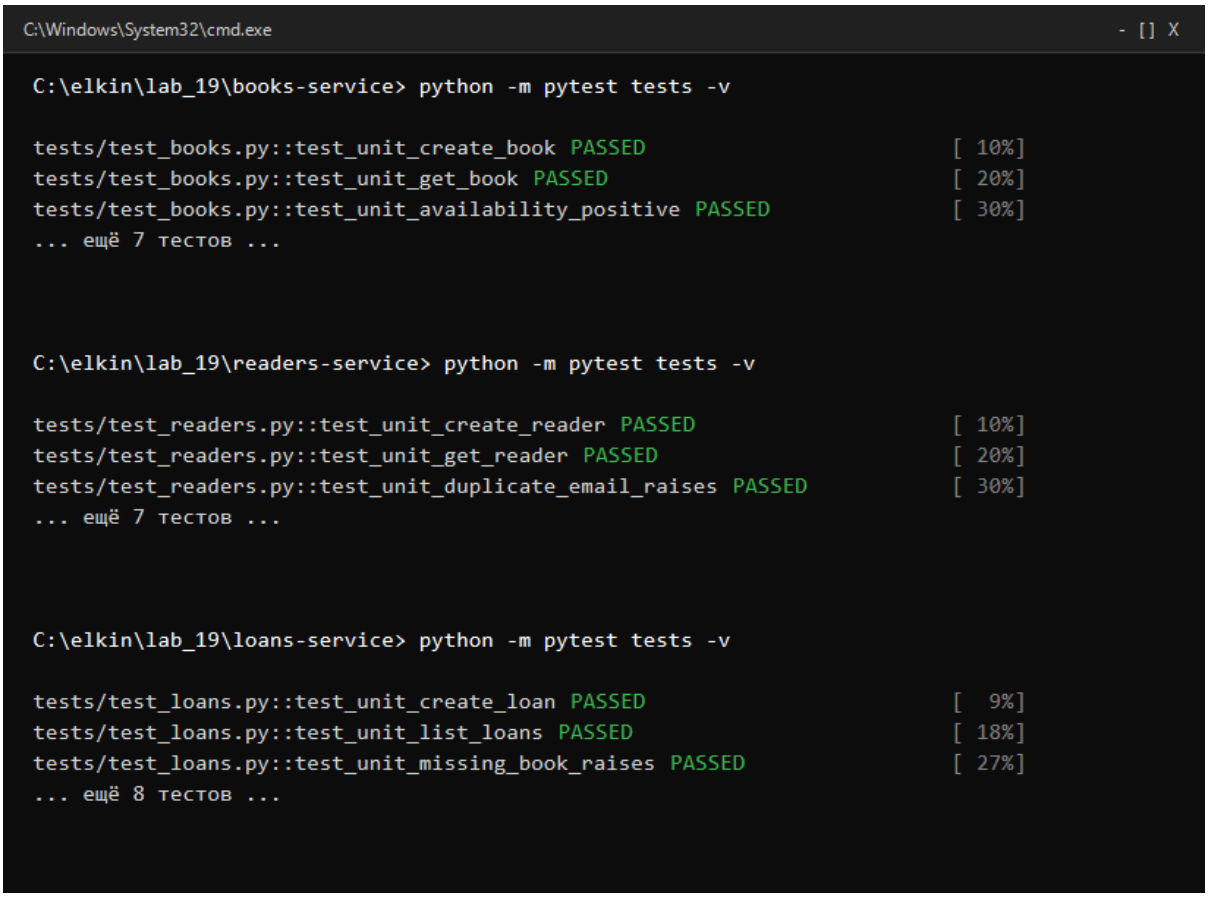


Рисунок 19.1 — Результат прохождения тестов трёх микросервисов

Листинги ключевых фрагментов программы

Листинг 19.3 — Проверка доступности книги и выдача (loans-service/app/services/service.py)

```
def issue_loan(self, payload: LoanCreate) -> Loan:
    reader = self._check_reader(payload.reader_id)
    if reader.get("status") not in ("active", "ok"):
        raise ValidationError("Читатель не может получать книги")

    book = self._check_book(payload.book_id)
```

```
    if not book.get("available", True):
        raise ServiceUnavailableError("Книга недоступна")

    self._reserve_book(payload.book_id)

    loan = self.repository.create(
        Loan(id="", book_id=payload.book_id,
             reader_id=payload.reader_id,
             due_date=payload.due_date, status="active"))

    self.events.publish("loan_issued",
                        {"loan_id": loan.id, "book_id": loan.book_id,
                         "reader_id": loan.reader_id})
    return loan
```

Листинг 19.4 — Файловая шина событий (shared/event_bus.py)

```
class FileEventBus:
    def __init__(self, events_dir: str):
        self._events_dir = events_dir
        os.makedirs(events_dir, exist_ok=True)
        self._lock = threading.Lock()

    def publish(self, event_type: str, payload: dict) -> dict:
        event = {"id": uuid.uuid4().hex, "type": event_type,
                 "timestamp": time.strftime("%Y-%m-%dT%H:%M:%S"),
                 "payload": payload}
        path = os.path.join(self._events_dir, f"{event['id']}.json")
        with self._lock:
            with open(path, "w", encoding="utf-8") as fh:
                json.dump(event, fh, ensure_ascii=False, indent=2)
        return event

    def poll(self, since_event_id: str | None = None) -> list[dict]:
        events = []
        for name in sorted(os.listdir(self._events_dir)):
            if not name.endswith(".json"):
                continue
            with open(os.path.join(self._events_dir, name),
                      "r", encoding="utf-8") as fh:
                event = json.load(fh)
            if since_event_id is None or event["id"] > since_event_id:
                events.append(event)
        return events
```

Листинг 19.5 — Фрагмент docker-compose.yml

```
services:
  readers:
    build: {context: .}
    working_dir: /app/readers-service
    environment:
      PORT: "8001"
      READERS_DB: /app/data/readers.json
    command: ["uvicorn", "app.main:app", "--host", "0.0.0.0",
              "--port", "8001"]
    volumes: [library-data:/app/data]
    networks: [library_net]
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; ..."]
      interval: 10s
      retries: 5
```

Вывод

В ходе выполнения лабораторной работы спроектирована и реализована микросервисная система библиотеки, состоящая из четырёх независимо запускаемых контейнеров: трёх предметных сервисов и API Gateway. Каждый сервис получил собственную зону ответственности, REST API и отдельное хранилище, что соответствует паттерну Database per Service. Синхронное взаимодействие реализовано через устойчивый HTTP-клиент с повторами и предохранителем, асинхронное — через файловую шину событий по модели Transactional Outbox. Gateway обеспечивает единую точку входа, агрегацию сценариев и проверку состояния системы, а Docker Compose — согласованный запуск и контроль работоспособности. Эксперимент с остановкой сервиса подтвердил, что система корректно деградирует и восстанавливается без ручного вмешательства.